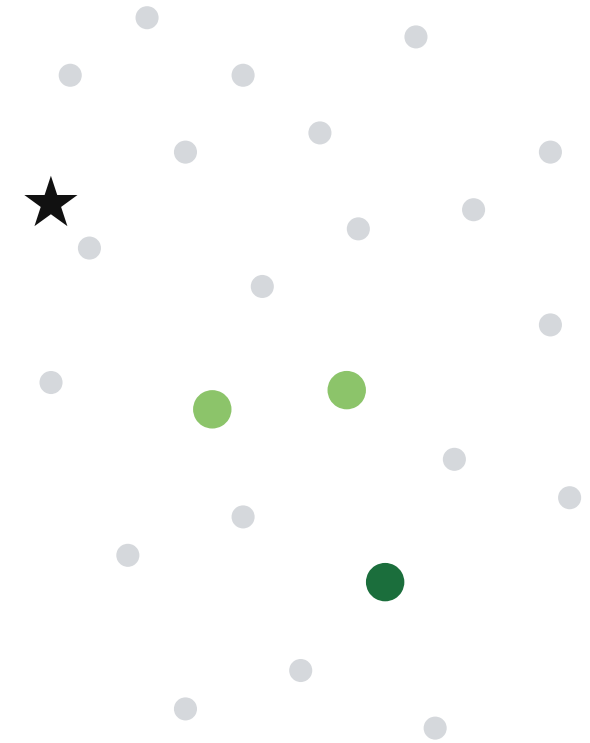


UMAP

Uniform Manifold Approximation and Projection

Seeing a 1536-dimensional product catalogue in two dimensions

Case study: hybrid semantic search over the e-commerce catalog (Dr. Sheth skincare)



● The problem: the retriever sees a space we cannot

Every product is a point in a space with more axes than we can draw

- Each product's title, description, tags and type are encoded into one dense vector.
- Similarity is a geometric fact in that space: cosine distance between two vectors.
- The search engine ranks by that geometry; a human cannot inspect 1536 axes.
- We need a 2-D picture that is faithful enough to reason about, and honest about what it distorts.

768 – 1536 dims

[0.021, -0.114, 0.308, 0.007, -0.256, ...]



2 dims

a scatter plot a human can read

Dimension reduction is lossy by construction ; the question is only which structure we choose to keep.

● Before UMAP (1): PCA

Linear, global, fast, and blind to fine structure

What it asks

- Which orthogonal linear directions carry the most variance?
- Project onto the top two; keep the axes that explain the catalogue as a whole.

What it gives us

- Global structure, to scale: the gap between two blobs is a real quantity.
- Deterministic and cheap; no tuning.

Where it fails us

- A linear projection: 1536 → 2 discards every non-linear direction.
- Embedding manifolds are curved, since semantic neighbourhoods are not planes.
- Everything inside a blob collapses: a niacinamide serum and a vitamin-C serum land on the same dot.
- Good for the macro split (face / hair / baby); useless for 'which SKUs are interchangeable'.

On our catalogue PCA correctly separates face, hair and baby ranges, because category vocabulary is the largest source of variance.

● Before UMAP (2): t-SNE

Solved locality, at the cost of global meaning and speed

What t-SNE fixed

- Non-linear: converts distances into neighbour probabilities, so curved manifolds survive.
- Matches high-D and low-D neighbour distributions by minimising KL divergence.
- Heavy-tailed Student-t in 2-D relieves the crowding problem, so clusters visibly separate.

What it still costs

- Inter-cluster distance is meaningless: KL barely penalises distant pairs.
- Perplexity is fragile: on small samples early exaggeration over-separates a few tight balls and scatters the rest.
- Euclidean only in scikit-learn, not the cosine metric our retriever actually scores with.
- Slow and memory-hungry; no reusable model to embed new points.

This is precisely the artefact we observed on the Dr. Sheth sample: large within-cluster spread, small between-cluster gaps: a property of the algorithm at that n , not of the products.

● UMAP: the idea in one sentence

McInnes, Healy & Melville, arXiv:1802.03426 (2018)

Assume the data lie on a manifold that is uniformly sampled; build a fuzzy topological representation of that manifold in high dimensions, then find the low-dimensional layout whose fuzzy representation is as close as possible.

1 Neighbours

Find the k nearest points of every point under cosine distance.

2 Fuzzy graph

Rescale distances per point, then symmetrise into one weighted graph.

3 Layout

Optimise a 2-D embedding: pull edges together, push sampled non-edges apart.

Grounded in Riemannian geometry and algebraic topology, but every step below is concrete and implementable.

● Step 1: local neighbourhoods, made comparable

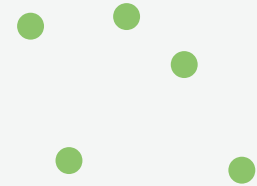
k-nearest neighbours, then a per-point rescaling of distance

- For each product, find its k nearest neighbours (our config: $k = 15$) by cosine distance.
- `local_connectivity = 1`: the nearest neighbour is always given distance ≈ 0 , so every point is connected to something.
- A per-point σ is fitted so the neighbour weights sum to $\log_2(k)$: the same 'amount of neighbourhood' for every point.
- Consequence: a dense region and a sparse region are treated identically. UMAP deliberately normalises density away.

dense region



sparse region



after per-point rescaling

both become:

“1 neighbour at distance ≈ 0 ,
15 neighbours within weight 1”

So cluster tightness on the plot is not a density measurement.

● Step 2: from many local views to one global graph

Fuzzy union: the step where global structure is actually created

- Each point's rescaled neighbourhood is a directed weighted graph: a local, one-sided view.
- A may call B a neighbour with weight 0.9 while B calls A a neighbour with weight 0.4.
- UMAP symmetrises with a fuzzy union (`set_op_mix_ratio = 1.0`): $w = a + b - ab$.
- Overlapping local views are stitched into a single connected structure covering the whole catalogue.

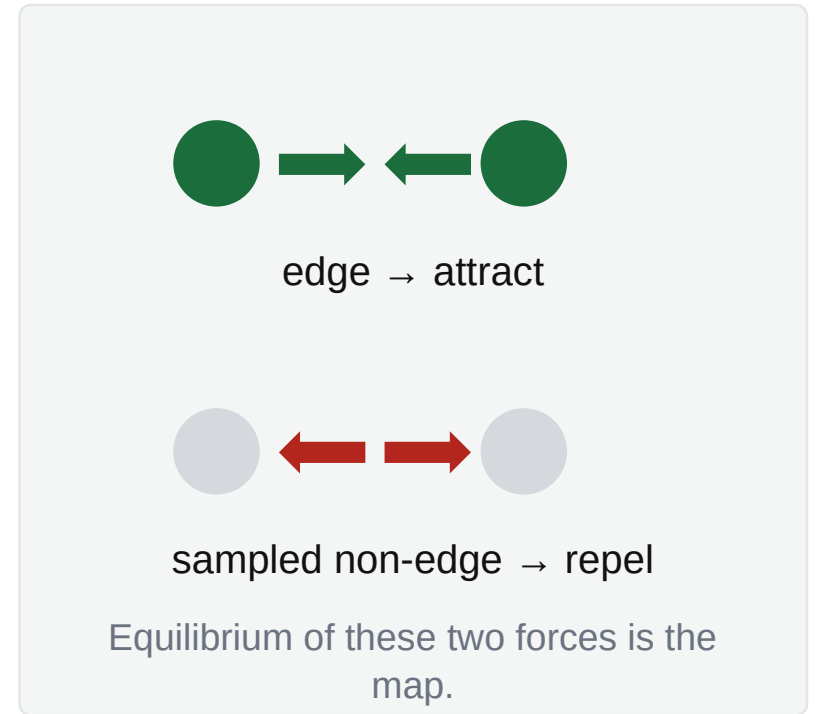


A never compares itself to G.
The chain of overlapping neighbourhoods places them correctly anyway, the way linked towns reveal the shape of a country.

● Step 3: laying the graph out in 2-D

Cross-entropy between the high-D fuzzy graph and a low-D one

- Initialise with a spectral embedding of the graph (init = 'spectral'): already a sensible global arrangement, not random noise.
- Attraction: every edge pulls its two points together, in proportion to its weight.
- Repulsion: for each edge, a few random non-neighbours are sampled and pushed away (negative_sample_rate = 5) so the layout cannot collapse.
- min_dist = 0.1 sets a floor on how tightly a clump may pack, a purely aesthetic constraint.
- Optimised by stochastic gradient descent over n_epochs (500 for $n < 10\,000$).



● What is preserved, and what is not

Read the map for membership, not for measurement

● Preserved: trust it

- Who is whose neighbour: the k-NN relation is the objective being optimised.
- Cluster membership: which SKUs the retriever treats as near-duplicates.
- Coarse connectivity: which groups the graph links through intermediate points.

● Not preserved: do not read it

- Distance between two separate blobs: largely a layout artefact.
- The size or tightness of a blob: density was normalised away in Step 1.
- Absolute orientation and axis meaning: the axes carry no units at all.

Global structure emerges only through chains of overlapping neighbourhoods ; it is qualitative, never metric.

● Why UMAP is fast

Four design decisions, each removing an $O(N^2)$ cost

1 Approximate neighbours

Nearest-neighbour descent finds the k -NN graph in roughly $O(N^{1.14})$ instead of comparing all pairs.

2 A sparse graph

Only $\sim N \times k$ edges are ever stored or optimised; t-SNE must model the full pairwise distribution.

3 Negative sampling

Repulsion is estimated from a handful of random non-neighbours per edge rather than every non-neighbour.

4 Plain SGD

Edge updates are independent, so the layout parallelises, unless `random_state` is set, which forces one thread.

In practice: minutes for millions of points, and a reusable `transform()` for new products, which t-SNE has no equivalent of.

● The two parameters that decide what you see

`n_neighbors` sets the scale of the question; `min_dist` sets the look of the answer

Parameter	Small value	Large value	Our setting
<code>n_neighbors</code>	5: very local; many tight, fragmented groups; fine detail, unreliable global shape	50–100: wider neighbourhoods; broad structure preserved, fine distinctions blurred	15
<code>min_dist</code>	0.0: points may pile up; maximum apparent clustering	0.5–0.99: evenly spread; topology kept, clumping suppressed	0.1 (default)
<code>metric</code>	cosine: matches how the retriever scores	euclidean: sensitive to vector magnitude	cosine
<code>random_state</code>	unset: parallel, faster, layout changes each run	fixed: reproducible, single-threaded, slower	42

Neither parameter has a 'correct' value: each answers a different question about the same catalogue.

● Our implementation

projection.py: what the visualiser actually runs

```
reducer = umap.UMAP(  
    n_components = dims,          # 2 or 3  
    metric       = "cosine",  
    random_state = 42,  
    n_neighbors  = min(15, max(2, n - 1)),  
)
```

- cosine matches es_queries.py, so UMAP is the only projection whose geometry agrees with the retriever.
- The query vector is stacked on top of the sampled catalogue vectors and fitted with them, so the ★ is placed inside the manifold, not transformed afterwards.
- Sample: up to max_docs = 1500 documents with a text_vector, scrolled in pages of 500.

Everything else is library default

```
min_dist = 0.1  
spread = 1.0  
n_epochs = 500 (n < 10 000)  
init = "spectral"  
local_connectivity = 1.0  
set_op_mix_ratio = 1.0  
negative_sample_rate = 5
```

random_state = 42 pins the layout but forces single-threaded execution, the main cost at large max_docs.

● What one point on the map actually is

A single indexed product document → one vector → one dot

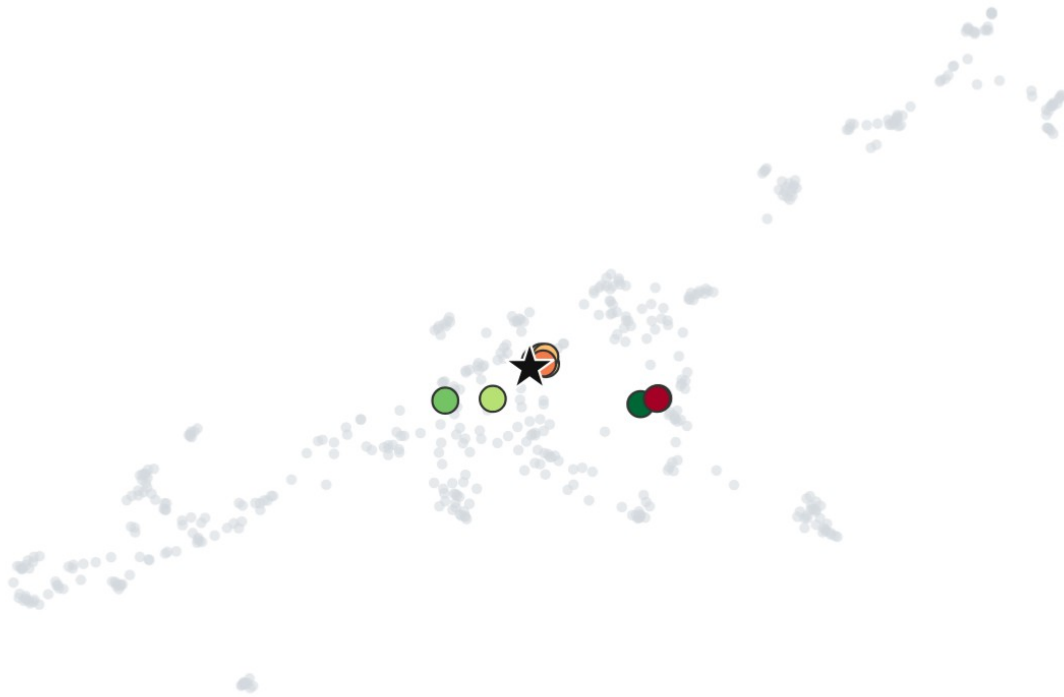
Dr. Sheth's Rosehip & Retinol Serum, 30 ML

product_type	Face Serum
vendor	Riaan Trading Company
tags	Dr Sheth, face serum, Retinol, Rosehip, serum
category	Skin Care > Face Serums
price	1118
description	"...antioxidants target dullness & uneven texture, boost collagen, fade dark spots..."

- title + description + tags + product_type are concatenated and encoded into text_embedding.
- So the dominant axis of variation is category language: serum vs cleanser vs sunscreen vs shampoo.
- A strong secondary axis is vendor vocabulary: repeated brand boilerplate makes one brand's products mutual neighbours.
- Price, inventory and images are not embedded ; they never influence a point's position.

● The search map

UMAP · 2-D · cosine · the ★ is the shopper's query vector



How to read it

- Grey dots: the sampled catalogue.
- Coloured dots: the ten products the engine returned, shaded by rank.
- ★: the query, fitted inside the same manifold.
- Clumps of grey are genuine neighbourhoods: variants and near-duplicates of one SKU.

Reproduce: /search-visualize/?query=<term>&method=umap&dims=2&color_by=product_type&max_docs=1500

● Reading this map

The query lands inside the neighbourhood it retrieved from

- The star sits among its own results. The query vector is fitted with the catalogue sample, and it comes to rest inside the region whose products share its vocabulary. That is what a healthy semantic match looks like.
- The ten results are not one blob but two nearby groups on the same filament: the query matches two adjacent product neighbourhoods rather than a single tight family.
- Rank colour and plotted position do not agree perfectly. The best and the worst of the ten sit side by side on the right. Ranking is decided by hybrid scoring in full dimension; position is decided by the neighbour graph. The two answer different questions.
- Read the map as: which neighbourhoods did the engine reach into, and did it miss an obvious one nearby.

★ query

● best match

● worst of the ten

● not returned

Colour encodes rank.

Position encodes neighbourhood.

The two need not agree.

● Why the map has this shape

The layout comes from the embedding space, not from the query

- The query is one extra point fitted alongside roughly 1500 catalogue vectors. It cannot bend the manifold. Remove it and the grey structure is essentially unchanged.
- The query decides only which dots are coloured. Every dot's position was already fixed by the neighbour graph of the catalogue itself.
- What builds the clusters is shared vocabulary, and the embedding text is title, description, tags and product_type. So the grouping is lexical before it is conceptual.
- The long thin filaments, rather than round balls, come from the fuzzy union: cleanser, face wash and toner form a continuum of overlapping descriptions, so they are stitched into a chain instead of a blob.

The hierarchy you are seeing

● Category

face care vs hair vs baby

generic brand and category words

● Product type

serum vs cleanser vs sunscreen

ingredient and benefit vocabulary

● SKU variants

30 ML vs 50 ML of one product

near identical text, stacked

Verify by recolouring: `color_by=product_type` should paint contiguous filaments; `color_by=vendor` should not, unless brand boilerplate is dominating the embedding.

● The same catalogue under all three methods

Dr. Sheth: cleansers, sunscreens, serums, moisturisers, plus hair and baby SKUs

PCA

Face / hair / baby macro-clusters.

Correct and to scale. Category vocabulary is the largest global variance direction, so PC1–PC2 land on exactly that split. Everything inside a blob is smeared together.

Trust for the macro split.

UMAP

Brand and product sub-clusters.

Resolves what PCA smeared: sunscreens together, cleansers together, variants of one SKU stacked. Brand blobs appear because vendor boilerplate makes descriptions mutual neighbours.

Trust for membership, never for gaps.

t-SNE

A few tight balls, the rest scattered.

A perplexity artefact at this sample size, not a property of the catalogue, and it runs on euclidean, so long descriptions drift outward on magnitude alone.

Discard at this catalogue size.

Colour by product_type to check the face/hair/baby claim, then by vendor: if a brand forms its own island, brand boilerplate is dominating the embedding.

● What UMAP still does not solve

Known limitations, and one that is ours rather than the algorithm's

● Inter-cluster distance stays unreliable

Better behaved than t-SNE, but still not a metric. Never quote a gap as a number.

● Density is discarded by design

The per-point rescaling means blob size says nothing about how many products live there.

● Stochastic and parameter-sensitive

Different seeds or `n_neighbors` give visibly different maps of identical data.

● It can invent structure

On small or noisy samples the optimiser will still produce clean-looking clusters. Verify against the data.

● Our sampling is not random

`sample_vectors` takes the first N documents in scroll order, not a random sample and not query-relevant. On a multi-merchant index larger than `max_docs` the map may show one merchant's slice.

What to take away

- UMAP builds a fuzzy neighbour graph and lays it out, so the map is a statement about who is near whom, and nothing more.
- Global structure is real but qualitative: it emerges from chains of overlapping neighbourhoods, not from measured distance.
- It is fast because it approximates three things: neighbours, the graph, and repulsion.
- For our catalogue: PCA for the macro split, UMAP for near-duplicate membership, t-SNE not at this size.
- The map is a debugging instrument for the retriever: read it as evidence, not as proof.

Reference

L. McInnes, J. Healy and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” arXiv:1802.03426 [stat.ML], 2018. · Implementation: umap-learn.